

How to fix a bad git tree object

Asked 14 years, 8 months ago Modified 1 year, 9 months ago Viewed 40k times



I have a local copy of a repository that not longer have any remote associated to it. I'm trying to push this repo into a new remote however everytime I get this message:

39



```
error: Could not read 9eefe9305253b2c039a54cbc8aa22f7f8e6e8790
fatal: bad tree object 9eefe9305253b2c039a54cbc8aa22f7f8e6e8790
```



I read in similar questions here that one way to fix it is retrieving this object from other copies of the repository or doing a hard reset. I can't do any of both since I don't have another copy of this repo.

Is there a way to simply remove this commit or some other kind of solution that will allow me to push the repo to the new remote keeping history?

git

version-control

repository

Share Improve this question Follow

asked Jan 25, 2012 at 15:58



Flupkear

2,301 7 30 32

- 1 You don't need a remote or "another copy of this repo" to do a `git reset --hard .` – [vcsjones](#) Jan 25, 2012 at 16:05

Thanks, didn't know that. I just did it but the problem still exists, the commit that's giving problems it's a very old one. – [Flupkear](#) Jan 25, 2012 at 16:18

two ways: one - look at the file directly (unzip the contents) and see if you can fix it. two - find the commit that references the tree in question and do a hard reset to the parent of that commit. Depending on which commit it is that references the concerning tree, you might lose a lot of data. – [Max Leske](#) Jan 25, 2012 at 16:19

if the commit is so old, you will lose **all** of the history that comes after that commit (it will still be in the repository of course but hard to access). So: do you need the newer commits? – [Max Leske](#) Jan 25, 2012 at 16:20

I would like not to loose newer commits history. The problem seems to be that the file in question is missing, I mean I searched for 9eefe9305253b2c039a54cbc8aa22f7f8e6e8790 in the git folder and it's simply not there. – [Flupkear](#) Jan 25, 2012 at 16:24

7 Answers

Sorted by: Highest score (default)





27



I ran into this and I really did not want to delete my `.git` and start over. No other suggestions I found on the internet helped me (including some pretty complicated `.git` surgery attempts).

The thing that fixed it for me was running:

```
git fetch --refetch
git gc --aggressive
```

You can confirm the fix with:

```
git fsck
```

[Share](#) [Improve this answer](#) [Follow](#)

answered Nov 2, 2022 at 8:38



[hauntsaninja](#)

1,094 10 12

3 Comments ^

Add a comment



[odinho - Vermont](#) Over a year ago

Not sure when `--refetch` was added or removed, but I do not have it in my Git 2.34.1 .

▲ 0

Reply



[hauntsaninja](#) Over a year ago

Looks like it was added in 2.36

▲ 2

Reply



[Sebastian Budka](#) Apr 15 at 10:19

I tried dozen solutions and only `git fetch --refetch` helped,, thanks!

▲ 0

Reply



7



Just Quit the application with which you accessing git and restart . Worked for me for xcode-iOS.

[Share](#) [Improve this answer](#) [Follow](#)

answered Oct 15, 2014 at 12:28



[Ankish Jain](#)

11.4k 6 39 34

3 Comments ^

Add a comment



[BadHorsie](#) Over a year ago

This actually worked for me using SourceTree. I just restarted the application and then I was able to push to the remote.

▲ 0

Reply



[Michael Radionov](#) Over a year ago

Same, it worked by restarting a terminal (iTerm2).

▲ 0

Reply



[l33t](#) Over a year ago

Same, it worked by restarting Github Desktop.

▲ 0

Reply



4



To actually fix the problem and not lose any data (provided that that tree is the only missing object, which I doubt) you could try this:

1. checkout the parent commit of the commit with the concerning tree
2. try **git cat-file -p** with the name of the problematic commit to see what the commit message says (hopefully it will tell you what changed)
3. now you might be able to determine the changes that were made and from this the directory structure can hopefully be inferred.
4. if 3 worked, then you can create your tree manually using a text editor and a zlib compressor. The entries in the tee file will be other tree objects or blobs. Hopefully most of the files and folders are shared (have no changes) between the two commits. This will allow you to reuse most of the entries from the tree object of the checked out commit.

Share Improve this answer Follow

edited Mar 25, 2019 at 23:34



[Adam Grant](#)

13.2k 10 63 65

answered Jan 25, 2012 at 16:41



[Max Leske](#)

5,155 6 47 57

3 Comments ^

Add a comment



[Flupkear](#) Over a year ago

The problem is that the file is missing, when I ran `git cat-file -p ...` I get this: error: unable to find 9e108293be3997c2d922b4c473a5a28cc737e446

▲ 1

Reply



[Max Leske](#) Over a year ago

Well, you could look into the commit manually... but you'll most likely have to write a script that will do that for you. Checkout this page for an overview of how the file format looks like: book.git-scm.com/1_the_git_object_model.html

▲ 0 Reply ...



Max Leske Over a year ago ✎

Well, the format of a tree object is pretty straight forward, so if you know what you are doing, you can create a file with the right bytes in it (you'll need all the signatures of the entries of course to do that). I'm not saying that it's a good (or even feasible) solution but it's a possibility to consider if the data is precious.

▲ 0 Reply ...



0



I had this problem. Restarting my computer actually fixed it so please try it first before doing anything else. Or maybe I just got lucky but worth a shot!

Share Improve this answer Follow

answered Jul 19, 2012 at 3:00



axsuul

7,510 9 60 73

Comments

Add a comment



0



I have my git repo in a directory which is also in the OneDrive folder on my computer. If you use OneDrive on another machine, push from the repo on that computer and then pull/push from your own machine again.

Share Improve this answer Follow

answered Mar 24, 2015 at 1:10



userFog

11.7k 1 19 7

Comments

Add a comment



0

None of these worked for me, since the object itself was gone.

I found a way to go up the tree and delete every object that was referencing this one, and then find the top-most commit and remove said commit from `.git/packed-refs`. It could also have



been inside `.git/refs/heads/your-branch`, but mine were in the packed version.



But there is a much faster way: `git fsck --name-objects`, that should tell you the dangling object and its parent (as I'd already found), but also name the object allowing you to more quickly find the broken commit to remove.

This answer is a good way to clear the rest of the backlog afterwards (it'll fully delete it, so make sure you're okay with that): <https://stackoverflow.com/a/69639136/179978>

Share Improve this answer Follow

answered Oct 24, 2023 at 11:27



[odinho - Vermont](#)

21.7k 6 44 34

Comments

Add a comment



None of the above answers helped me. I got into a bad state when I tried amending a commit while `git gc` was running simultaneously (bad idea, apparently :)).

0



What worked for me was backing up the contents of a file I was editing, running `git reset --hard <previous_commit_sha>`, then restoring the contents of a file and making a new commit.



Share Improve this answer Follow

answered Dec 10, 2024 at 16:33



[Dmitry Pashkevich](#)

13.6k 10 59 75

Comments

Add a comment

Start asking to get answers

Find the answer to your question by asking.

[Ask question](#)

Explore related questions

[git](#) [version-control](#) [repository](#)

See similar questions with these tags.

